

International Cybersecurity and Digital Forensics Academy (ICDFA)**HUSSEINI SULEIMAN****2025/FWSD/11478**fwsd2511478@students.icdfa.edu.ng**DATE: 17TH September, 2026****SBT-DF203: Basic Networking Skills for Digital Forensics****LAB 6: Firewall Traffic Control and Forensic Verification****Three-Week Delivery Block 2/3 | 12-18 September 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 6
Lab Title	Firewall Traffic Control and Forensic Verification
Delivery Block	2/3 of 3
Scheduled Dates	12-18 September 2026
Estimated Duration	4-5 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	Baseline and blocked HTTP PCAPNG files; iptables ruleset exports

1. Source Materials and Slide Alignment

- Firewall: network-based and host-based firewalls, iptables chains and rules, ACCEPT/DROP actions, UFW relationship, NFQUEUE concept and HTTP blocking.
- The slide practical lists rules, adds/removes an INPUT rule, uses a local Apache page and observes allowed versus blocked hosts.
- The homework requires traffic generated by curl from a different machine and forensic observation of the result.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

You are validating whether a Linux host-based firewall correctly blocks HTTP traffic from one authorized lab client while allowing another. You will capture baseline traffic, apply a narrowly scoped iptables rule, capture the blocked connection behaviour, prove the rule hit using counters, remove the rule and document the evidential differences.

3. Learning Outcomes

- Explain the purpose of host-based and network-based firewalls.
- Interpret INPUT, OUTPUT and FORWARD chains.
- Create, verify and remove a narrowly scoped iptables rule.
- Capture allowed and blocked HTTP connection attempts.
- Correlate packet evidence with firewall rule counters and application behaviour.
- Explain DROP versus REJECT and identify retransmission/timeout evidence.
- Restore the original ruleset safely.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.

- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- Before making changes, export the existing ruleset. Apply only a source-specific rule for the assigned lab IP. Never flush all firewall rules on a remotely administered or production system.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Server VM	Kali/Ubuntu with Apache	Protected host.
Client VM	Linux or Windows on host-only network	Generate HTTP requests.
Firewall tools	iptables, iptables-save	Apply and preserve rules.
Capture tools	tshark/Wireshark	Capture baseline and blocked traffic.
Client tools	curl, browser	Test access.
Optional extension	NetfilterQueue	Instructor-led packet interception demonstration.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-  
Lab6/{evidence,working,exported,reports,screenshots,scripts}  
cd ~/SBT-DF203-Lab6  
pwd  
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```

sudo apt update
sudo apt install -y apache2 curl iptables tshark wireshark
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall
Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>\n' | sudo tee
/var/www/html/firewall_lab.html
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee
reports/iptables_before_sha256.txt

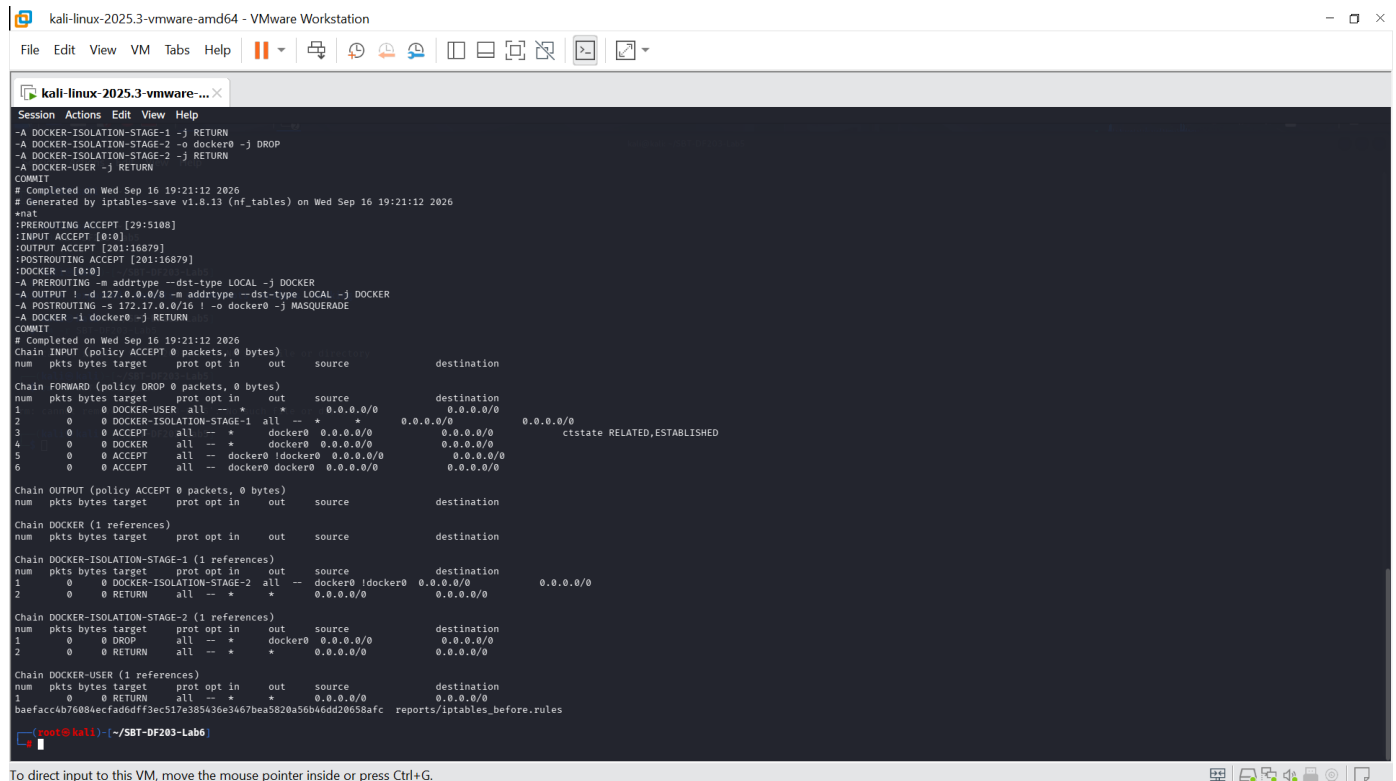
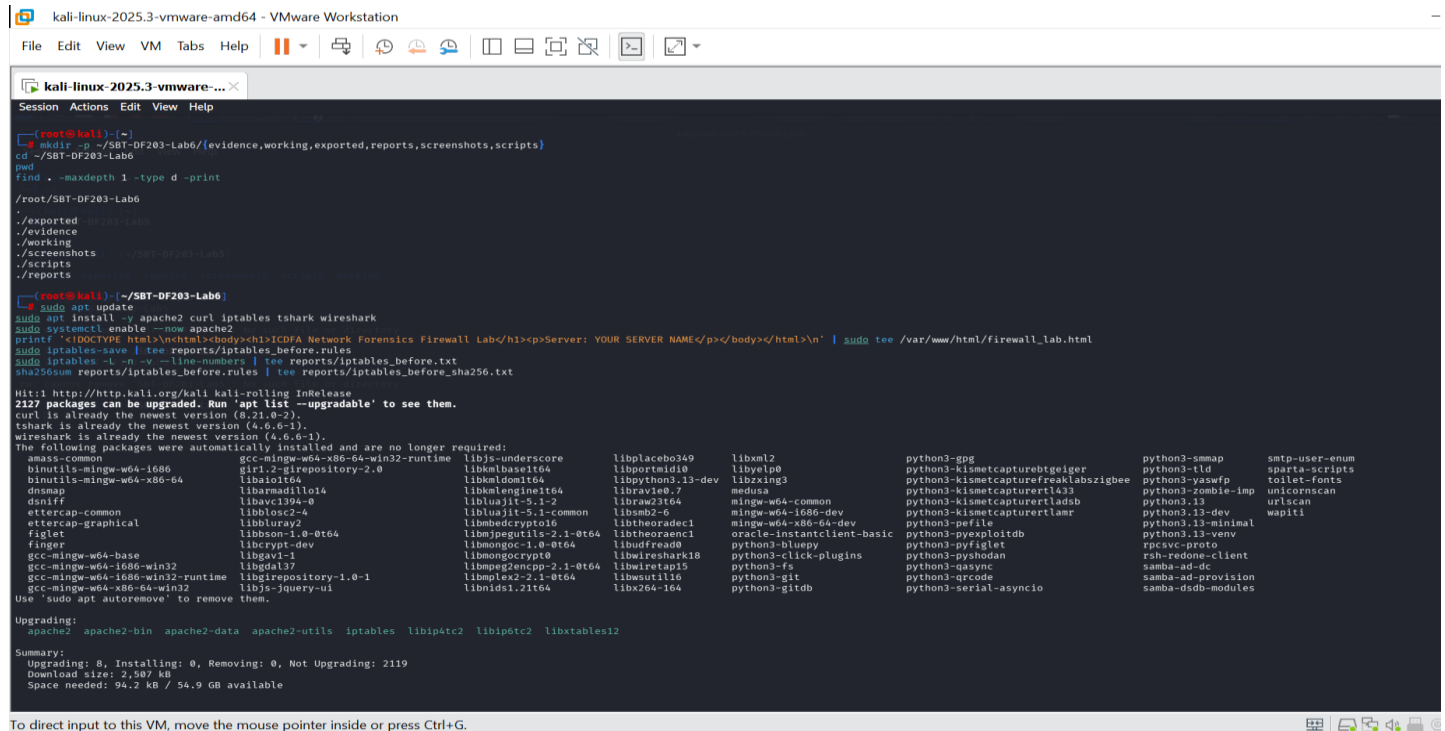
```

Screenshot required: Original firewall ruleset, rule listing, hash and the training webpage.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab6-YourFullName
Trainee name	HUSSEINI SULEIMAN
Date and time started	17 TH SEPTEMBER, 2026
Evidence file name(s)	SBT-DF203-Lab6
Source or generation method	ROOT@KALI
Original SHA-256	baefacc4b76084ecfad6dff3ec517e385436e3467bea5820a56b46dd20658afc
Working-copy SHA-256	
Analysis workstation/V M	

Field	Student Entry
Notes on any changes	



8. Part A - Record the Lab Network and Baseline Access

On the server

```
ip -br address | tee reports/server_interfaces.txt
ip route | tee reports/server_routes.txt
sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
```

On the client, replace SERVER_IP

```
curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html 2>&1 | tee
baseline_curl.txt
```

Record the server IP, allowed client IP, blocked client IP, interface and default gateway. Take a snapshot before changing the firewall.

Answer: see the server detail: UNKNOWN 127.0.0.1/8 ::1/128

eth0 UP 192.168.142.128/24

fe80::fe8d:6b5d:6ee4:ee1b/64

docker0 DOWN 172.17.0.1/16

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
Session Actions Edit View Help
root@kali:~/SBT-DF203-Lab6#
# On the server
ip -br address | tee reports/server_interfaces.txt
ip route | tee reports/server_routes.txt
sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt

# On the client, replace SERVER_IP
curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html 2>&1 | tee baseline_curl.txt

to UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.142.128/24 fe80::fe8d:6b5d:6ee4:ee1b/64
docker0 DOWN 172.17.0.1/16
default via 192.168.142.2 dev eth0 proto dhcp src 192.168.142.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.142.0/24 dev eth0 proto kernel scope link src 192.168.142.128 metric 100
LISTEN 0 511 *:* users:((("apache2",pid=37834,fd=4),("apache2",pid=37833,fd=4),("apache2",pid=37831,fd=4),("apache2",pid=37829,fd=4),("apache2",pid=37818,fd=4))
* Resolving timed out after 5003 milliseconds
* closing connection #0
curl: (28) Resolving timed out after 5003 milliseconds

root@kali:~/SBT-DF203-Lab6#
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

9. Part B - Capture the Allowed HTTP Baseline

```
# On server, replace IFACE and CLIENT_IP
sudo tshark -i IFACE -f 'host CLIENT_IP and tcp port 80' -a duration:30 -w
evidence/http_allowed.pcapng &
sleep 3
# Run curl from client during this capture
wait
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
```

10. Part C - Apply and Verify the Blocking Rule

Use the exact source IP of the assigned blocked client. Insert the rule near the top of INPUT so it is evaluated before a broad ACCEPT rule.

```
BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
printf 'Rule verified present.\n' | tee reports/rule_verification.txt
```



```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-... X
Session Actions Edit View Help
root@kali: ~/SBT-DF203-Lab6
BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -I INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
printf 'Rule verified present.\n' | tee reports/rule_verification.txt

Chain INPUT (policy ACCEPT 5 packets, 328 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0         tcp dpt:80
2      0      0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0         tcp dpt:80
Rule verified present.

root@kali: ~/SBT-DF203-Lab6

```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

11. Part D - Capture Blocked Traffic and Rule Counters

On server

```

sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w
evidence/http_blocked.pcapng &
sleep 3

```

From blocked client during capture:

```

# curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
wait

```

```

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt

```

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-... X
Session Actions Edit View Help
[~] (root@kali) [~/SBT-DF203-Lab6]
BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -I INPUT -n -y --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
printf 'Rule verified present.\n' | tee reports/rule_verification.txt

Chain INPUT (policy ACCEPT 5 packets, 328 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0      tcp dpt:80
2      0      0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0      tcp dpt:80
Rule verified present.

[~] (root@kali) [~/SBT-DF203-Lab6]
# On server
sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
sleep 3
# From blocked client during capture:
$ curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
wait

sudo iptables -I INPUT -n -y --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt

[!] 50902
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'IFACE'
tshark: There is no device named "IFACE".
(No such device exists)
[1] * exit 1 sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -
Chain INPUT (policy ACCEPT 11 packets, 738 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0      tcp dpt:80
2      0      0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0      tcp dpt:80
sha256sum: evidence/http_blocked.pcapng: No such file or directory

[~] (root@kali) [~/SBT-DF203-Lab6]

```

12. Part E - Compare Allowed and Blocked Captures

for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
 echo "===== \$PCAP ====="
 tshark -r "\$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response ||
 tcp.analysis.retransmission' \
 -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e
 ip.dst -e tcp.dstport \
 -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e
 http.response.code
 done | tee reports/allowed_vs_blocked.tsv

Indicator	Allowed Capture	Blocked Capture
Client SYN visible?		
Server SYN-ACK visible?		
Handshake completed?		
HTTP GET visible?		
HTTP response visible?		
Retransmissions/timeouts		

Indicator	Allowed Capture	Blocked Capture
iptables counter change		
curl result		

The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The terminal window displays the following commands and output:

```
sudo iptables -I INPUT -s "$BLOCKED_CLIENT_IP" -j DROP --dport 80 -i DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
printf "Rule verified present.\n" | tee reports/rule_verification.txt

Chain INPUT (policy ACCEPT 5 packets, 328 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 DROP     tcp  --  *      *        192.168.56.20      0.0.0.0/0          tcp dpt:80
2      0      0 DROP     tcp  --  *      *        192.168.56.20      0.0.0.0/0          tcp dpt:80
Rule verified present.

[root@kali]~/SBT-DF203-Lab6#
# On server
sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
sleep 3
# From blocked client during capture:
# curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
wait

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt

[1] 58902
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on "IFACE"
tshark: There is no device named "IFACE".
(No such device exists)
[1] * exit 1 sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -
Chain INPUT (policy ACCEPT 11 packets, 738 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 DROP     tcp  --  *      *        192.168.56.20      0.0.0.0/0          tcp dpt:80
2      0      0 DROP     tcp  --  *      *        192.168.56.20      0.0.0.0/0          tcp dpt:80
sha256sum: evidence/http_blocked.pcapng: No such file or directory

[root@kali]~/SBT-DF203-Lab6#
# for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "==== $PCAP ====="
tshark -e "$PCAP" -x 'tcp.flags.syn=1 || http.request || http.response || tcp.analysis.retransmission' \
  -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv

==== evidence/http_allowed.pcapng ====
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
tshark: The file "evidence/http_allowed.pcapng" doesn't exist.
==== evidence/http_blocked.pcapng ====
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
tshark: The file "evidence/http_blocked.pcapng" doesn't exist.

[root@kali]~/SBT-DF203-Lab6#
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

13. Part F - Remove the Rule and Restore Access

BLOCKED_CLIENT_IP=192.168.56.20

```
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP ||
```

```
echo 'Rule successfully removed'
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
```

Confirm access from the previously blocked client

```
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
```

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-...
Session Actions Edit View Help
# On server
sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
sleep 3
# From blocked client during capture:
# curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
wait

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt

[1] 50902
Running as user "root" and group "root". This could be dangerous.
Warnings: program compiled against libx1 215 using older 214
Capturing on "IFACE"
(No such device exists)
tshark: There is no device named "IFACE".
(No such device exists)
[1] * exit 1 sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -
Chain INPUT (policy ACCEPT 11 packets, 738 bytes)
num pkts bytes target prot opt in out source destination tcp dpt:80
1 0 0 DROP tcp -- * * 192.168.56.20 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
2 0 0 DROP tcp -- * * 192.168.56.20 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
sha256sum: evidence/http_blocked.pcapng: No such file or directory

(root@kali) ~/SBT-DF203-Lab6
# for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "==== PCAP"
tshark -i "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
  -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv

==== evidence/http_allowed.pcapng ====
Running as user "root" and group "root". This could be dangerous.
Warnings: program compiled against libx1 215 using older 214
tshark: The file "evidence/http_allowed.pcapng" doesn't exist.
==== evidence/http_blocked.pcapng ====
Running as user "root" and group "root". This could be dangerous.
Warnings: program compiled against libx1 215 using older 214
tshark: The file "evidence/http_blocked.pcapng" doesn't exist.

(root@kali) ~/SBT-DF203-Lab6
# BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
# Confirm access from the previously blocked client
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html

Chain INPUT (policy ACCEPT 22 packets, 1750 bytes)
num pkts bytes target prot opt in out source destination tcp dpt:80
1 0 0 DROP tcp -- * * 192.168.56.20 0.0.0.0/0 0.0.0.0/0 tcp dpt:80

(root@kali) ~/SBT-DF203-Lab6

```

14. Part G - Optional NFQUEUE Observation

Instructor-Led Extension

The slide introduces NFQUEUE and Python packet callbacks. Complete this only in the prepared lab VM because package names and Python bindings vary. The core assessed task is iptables rule evidence and PCAP verification.

- Explain that NFQUEUE passes matching packets to a userspace program.
- Identify where a callback can accept, modify or drop the packet.
- Document the rule, queue number, process and cleanup command.
- Never leave an NFQUEUE rule active without a listening process because traffic may stall.

15. Forensic Interpretation Questions

1. Why does DROP commonly cause SYN retransmissions and a timeout?
2. How would a REJECT rule differ in the packet capture and client output?
3. Why are firewall counters valuable corroborating evidence?
4. What evidence would show that the web server was down rather than firewall-blocked?
5. What is the risk of deleting a rule by line number after other rules have changed?

Required Screenshots Checklist

- ☐ Original ruleset and hash.
- ☐ Network addresses and Apache listener.
- ☐ Successful baseline curl.
- ☐ Allowed HTTP capture handshake/request/response.
- ☐ Inserted DROP rule and verification.
- ☐ Blocked curl timeout.
- ☐ Blocked capture showing retransmissions/no response.
- ☐ Rule packet/byte counters after test.
- ☐ Allowed-versus-blocked comparison.
- ☐ Rule removal and restored access.

Student Report Template

6. Executive summary.
7. Lab network and scope.
8. Original firewall state and evidence hashes.
9. Allowed baseline analysis.
10. Rule implementation and verification.
11. Blocked traffic analysis and counter correlation.
12. DROP versus REJECT discussion.
13. Cleanup and restored service.
14. Conclusions and appendix.

Submission Requirements

- One PDF report named SBT-DF203-Lab6_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Safety and ruleset preservation	15	Original rules exported; narrow rule; clean restoration.
Baseline capture	15	Complete allowed HTTP session.
Firewall rule execution	20	Correct insertion, verification, counters and removal.
Blocked traffic forensics	25	Accurate packet/curl/counter correlation.
Interpretation	15	DROP, REJECT, service-down and evidence limitations.
Report quality	10	Professional screenshots and narrative.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
INPUT chain	Traffic destined for the local host.
DROP behaviour	Packet silently discarded; client often retransmits then times out.
REJECT behaviour	Firewall sends an error/reset, producing faster failure.
Counter evidence	Packet/byte counters increase when the rule matches.
Allowed session	SYN, SYN-ACK, ACK, HTTP request and response.

Checkpoint	Expected Observation
Cleanup	Delete the exact rule specification and confirm access returns.

Command Reference Summary

Command	Purpose
<code>iptables -L -n -v --line-numbers</code>	List rules and counters.
<code>iptables-save</code>	Export the current ruleset.
<code>iptables -I INPUT 1 ... -j DROP</code>	Insert a narrow blocking rule.
<code>iptables -C ...</code>	Check whether a rule exists.
<code>iptables -D ...</code>	Delete an exact rule.
<code>tcp.analysis.retransmission</code>	Filter retransmitted TCP packets.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.